UBUNTU 22.04 LTS · DEV ENVIRONMENT

우분투 개발 환경 구축 가이드

Python 3.11 · 가상환경 · VS Code 연동 · OpenVINO 설치와 동작 검증

PART 1 Python 3.11 PART 2 가상환경 PART 3 VS Code PART 4 OpenVINO

CONTENTS

목차

PART 1

Python 3.11 설치

deadsnakes PPA로 우분투 22.04에 Python 3.11을 시스템 파이썬과 나란히 설치합니다.

05 – 09

PART 2

가상환경(venv) 만들기

프로젝트 전용 독립 공간을 만들고 활성화·비활성화하는 법을 익힙니다.

10 – 13

PART 3

VS Code 설치와 연동

VS Code를 설치하고 Python 확장을 넣어 방금 만든 가상환경과 연결합니다.

14 – 19

PART 4

OpenVINO 설치와 동작 검증

가상환경 안에 OpenVINO를 설치하고 버전·디바이스 확인까지 검증합니다.

20 – 25

BEFORE YOU START

사전 준비물

1 우분투 24.04 LTS

이 자료의 모든 명령어는 Ubuntu 24.04 LTS(Jammy Jellyfish) 기준입니다.

2 터미널과 sudo 권한

패키지 설치에는 관리자 권한이 필요합니다. sudo를 쓸 수 있는 계정으로 로그인하세요.

3 인터넷 연결

PPA, VS Code, PyPI 패키지를 내려받으려면 네트워크 연결이 필요합니다.

4 디스크 여유 공간

VS Code와 OpenVINO를 합쳐 최소 2~3GB 정도의 여유 공간을 권장합니다.

OVERVIEW

전체 흐름 한눈에 보기

01

Python 3.11 설치

deadsnakes PPA 추가 → python3.11 설치

→

02

가상환경 생성

python3.11 -m venv .venv → 활성화

→

03

VS Code 연동

VS Code 설치 → Python 확장 → 인터프리터를 .venv로 지정

→

04

OpenVINO 설치·검증

pip install openvino → 버전/디바이스 확인

PART

1

Python 3.11 설치

우분투 22.04의 기본 파이썬(3.10)은 그대로 두고, deadsnakes PPA로 Python 3.11을 나란히 설치합니다.

왜 3.11을 따로 설치하나

개념

우분투 기본 파이썬

- 24.XX LTS의 기본 python3는 3.11입니다.
- 시스템 도구들이 이 버전에 의존하고 있어 함부로 바꾸면 안 됩니다.
- apt 등 OS 유틸리티가 여기에 물려 있습니다.

deadsnakes PPA

- 우분투 공식 저장소 밖에서 최신 파이썬 버전을 제공하는 PPA입니다.
- python3.11을 기존 python3와 별도 이름으로 설치합니다.
- 시스템 파이썬을 건드리지 않아 안전합니다.

PPA 추가

설치

명령어	설명
`sudo apt update`	패키지 목록을 최신화합니다.
`sudo apt install -y software-properties-common`	add-apt-repository 명령을 쓰기 위한 도구를 설치합니다.
`sudo add-apt-repository ppa:deadsnakes/ppa`	deadsnakes PPA를 저장소 목록에 추가합니다.
`sudo apt update`	추가한 PPA의 패키지 목록을 반영합니다.

실행 명령어

설치

bash

```
$ sudo apt update  패키지 목록 최신화
$ sudo apt install -y software-properties-common  저장소 관리 도구 설치
$ sudo add-apt-repository ppa:deadsnakes/ppa  PPA 추가 (Enter로 확인)
$ sudo apt update  목록 반영
$ sudo apt install python3.11 python3.11-venv python3.11-dev  3.11 본체 + venv + 개발 헤더 설치
$ python3.11 --version  Python 3.11.x 출력 확인
```

직접 해보기

확인

TASK

python3.11 --version 으로 3.11.x가 나오는지 확인하고, python3 --version 으로 시스템 기본 버전(3.10)이 그대로인지도 비교해보세요.

TIP

기존 python3 명령은 그대로 3.10을 가리킵니다. 새로 설치한 3.11은 python3.11 이라는 이름으로 따로 존재하며, 서로 충돌하지 않습니다.

PART

2

가상환경(venv) 만들기

프로젝트마다 독립된 패키지 공간을 만들어, 서로 다른 프로젝트의 패키지 버전이 충돌하지 않게 합니다.

가상환경이 필요한 이유

개념

전역(글로벌) 설치

- pip install이 시스템 전체에 영향을 줍니다.
- 프로젝트마다 필요한 패키지 버전이 다르면 충돌이 납니다.
- 삭제나 정리가 지저분해지기 쉽습니다.

가상환경(venv)

- 프로젝트 폴더 안에 독립된 파이썬 공간을 만듭니다.
- 이 안에서 설치한 패키지는 이 프로젝트에만 적용됩니다.
- 필요 없어지면 폴더만 삭제하면 깨끗이 사라집니다.

실행 명령어

생성

bash

```
$ mkdir -p ~/openvino-project
$ cd ~/openvino-project  작업 폴더 생성 후 이동
$ python3.11 -m venv .venv  3.11 기반 가상환경 생성
$ source .venv/bin/activate  가상환경 활성화
$ python --version  활성화 상태에서는 python이 3.11을 가리킴
```

확인 · 비활성화

확인

명령어	설명
`which python`	지금 python이 어느 경로를 가리키는지 확인합니다.
`pip --version`	가상환경 안의 pip 버전과 경로를 확인합니다.
`deactivate`	가상환경을 빠져나와 원래 쉘로 돌아갑니다.

직접 해보기

확인

TASK

.venv 가상환경을 만들고 활성화한 뒤, which python으로 가상환경 안의 파이썬 경로가 맞는지 확인해보세요. 그다음 deactivate로 빠져나와보세요.

TIP

터미널 프롬프트 맨 앞에 (.venv) 표시가 붙으면 가상환경이 활성화된 상태입니다. 이 표시를 보는 습관을 들이면 실수를 줄일 수 있습니다.

PART

3

VS Code 설치와 연동

VS Code를 설치하고 Python 확장을 넣어, 방금 만든 가상환경을 에디터의 기본 인터프리터로 지정합니다.

실행 명령어

설치

bash

```
$ sudo snap install --classic code  VS Code 설치 (공식 스냅 패키지)
$ code --version  설치된 버전 확인
$ cd ~/openvino-project && code .  작업 폴더를 VS Code로 열기
```

NOTE apt 저장소를 직접 추가해 code 패키지로 설치하는 방법도 있지만, snap 한 줄이 가장 간단합니다.

Python 확장 설치하기

설정

1. **확장 패널 열기** — VS Code 왼쪽의 Extensions 아이콘을 클릭하거나 Ctrl+Shift+X를 누릅니다.
2. **'Python' 검색** — 검색창에 Python을 입력합니다.
3. **Microsoft 공식 확장 설치** — 게시자가 Microsoft로 표시된 Python 확장을 찾아 Install을 클릭합니다.
4. **필요 시 재시작** — 설치 후 재시작을 안내하는 알림이 뜨면 그대로 따라줍니다.

가상환경 인터프리터 선택하기

설정

1	명령 팔레트 열기	Ctrl+Shift+P를 눌러 명령 팔레트를 엽니다.
2	인터프리터 선택 명령 실행	Python: Select Interpreter 를 입력하고 선택합니다.
3	목록에서 .venv 선택	목록에 방금 만든 .venv가 보이면 선택합니다.
4	안 보이면 경로 직접 입력	Enter interpreter path를 선택해 .venv/bin/python 경로를 직접 입력합니다.

단축키는 배포판/키보드 설정에 따라 다를 수 있습니다. 다르다면 명령 팔레트 아이콘을 직접 눌러도 됩니다.

실행 명령어

확인

VS Code 내장 터미널

```
# 하단 상태 표시줄에 .venv (Python 3.11.x) 가 보이는지 먼저 확인
$ which python   경로가 프로젝트 폴더의 .venv를 가리키는지 확인
$ python --version   3.11.x가 출력되면 연동 성공
```

직접 해보기

확인

TASK

VS Code에서 새 .py 파일을 만들고 print("hello") 를 실행해, 하단 터미널과 상태 표시줄에 표시되는 인터프리터가 .venv인지 확인해보세요.

TIP

상태 표시줄의 인터프리터 이름을 클릭하면 언제든 다른 가상환경으로 바꿀 수 있습니다.

PART

4

OpenVINO 설치와 동작 검증

가상환경 안에 OpenVINO를 설치하고, 버전 확인과 사용 가능한 디바이스 확인까지 마쳐 정상 동작을 검증합니다.

OpenVINO란 무엇인가

개념

무엇을 하는 도구인가

- Intel이 만든 딥러닝 추론 최적화 툴킷입니다.
- 학습이 끝난 모델을 CPU 등 다양한 하드웨어에서 빠르게 실행하도록 돕습니다.
- PyPI를 통해 pip으로 간단히 설치할 수 있습니다.

이 실습에서 할 일

- 가상환경 안에 pip으로 OpenVINO를 설치합니다.
- 버전을 출력해 설치 자체가 됐는지 확인합니다.
- 사용 가능한 디바이스 목록을 조회해 실제 동작을 검증합니다
.

실행 명령어

설치

```bash (.venv)
# .venv가 활성화된 상태인지 프롬프트를 먼저 확인
$ pip install -U pip  pip 자체를 최신화
$ pip install -U openvino  OpenVINO 설치
```

NOTE 이 자료 작성 시점(2026년 8월) 기준 최신 버전은 2026.3이며, Python 3.10~3.14를 지원합니다. 설치 시점에 따라 버전은 달라질 수 있습니다.

실행 명령어

검증

bash (.venv)

```
$ python -c "import openvino as ov; print(ov.__version__)"  버전 확인

$ python -c "from openvino import Core; print(Core().available_devices)"  사용 가능한 디바이스 확인
# 예: ['CPU'] 처럼 장치 목록이 나오면 정상 동작입니다.
```

직접 해보기

검증

TASK

가상환경이 활성화된 상태에서 openvino를 설치하고, 버전 확인과 available_devices 확인까지 순서대로 실행해보세요.

TIP

GPU나 NPU가 없는 일반 PC라면 available_devices 결과에 CPU만 나와도 정상입니다. 목록이 비어 있지만 않으면 됩니다.

PART 5 · QUICK REFERENCE

전체 명령어 총정리

단계	명령어
Python 3.11 설치	`add-apt-repository ppa:deadsnakes/ppa · apt install python3.11`
가상환경 생성	`python3.11 -m venv .venv`
가상환경 활성화	`source .venv/bin/activate`
VS Code 설치	`snap install --classic code`
인터프리터 선택	`Ctrl+Shift+P → Python: Select Interpreter`
OpenVINO 설치	`pip install -U openvino`
버전 확인	`python -c "import openvino as ov; print(ov.__version__)"`
동작 검증	`python -c "from openvino import Core; print(Core().available_devices)"`

PART 5 · TROUBLESHOOTING

자주 만나는 문제 해결

! error: externally-managed-environment

시스템 파이썬에 pip로 직접 설치하려 할 때 나는 보호 오류입니다. 반드시 .venv를 활성화한 뒤 pip install을 실행하세요.

! VS Code에 .venv가 안 보일 때

Python: Select Interpreter에서 Enter interpreter path를 선택해 .venv/bin/python 경로를 직접 입력하거나, 프로젝트 폴더에서 VS Code를 다시 열어보세요.

! ModuleNotFoundError: openvino

VS Code 상태 표시줄의 인터프리터가 .venv인지 다시 확인하세요. 다른 파이썬 환경에 설치되었을 가능성이 높습니다.

! available_devices가 비어 있을 때

드물지만 설치가 꼬였을 수 있습니다. pip uninstall openvino 후 pip install -U openvino로 재설치해보세요.

PART 5 · SELF CHECK

완료 체크리스트

- [x] python3.11 --version 실행 시 3.11.x가 정상 출력된다
- [x] .venv 가상환경을 생성하고 활성화까지 확인했다
- [x] VS Code 상태 표시줄에 .venv (Python 3.11.x) 가 표시된다
- [x] 가상환경 안에서 pip install -U openvino가 정상 완료됐다
- [x] available_devices 결과에 CPU 이상이 출력되는 것을 확인했다

고맙습니다

환경 구축은 한 번만 잘 해두면 그다음부터는 순식간입니다.

문제가 생기면 24페이지의 '자주 만나는 문제 해결'을 먼저 확인해보세요.

Q & A